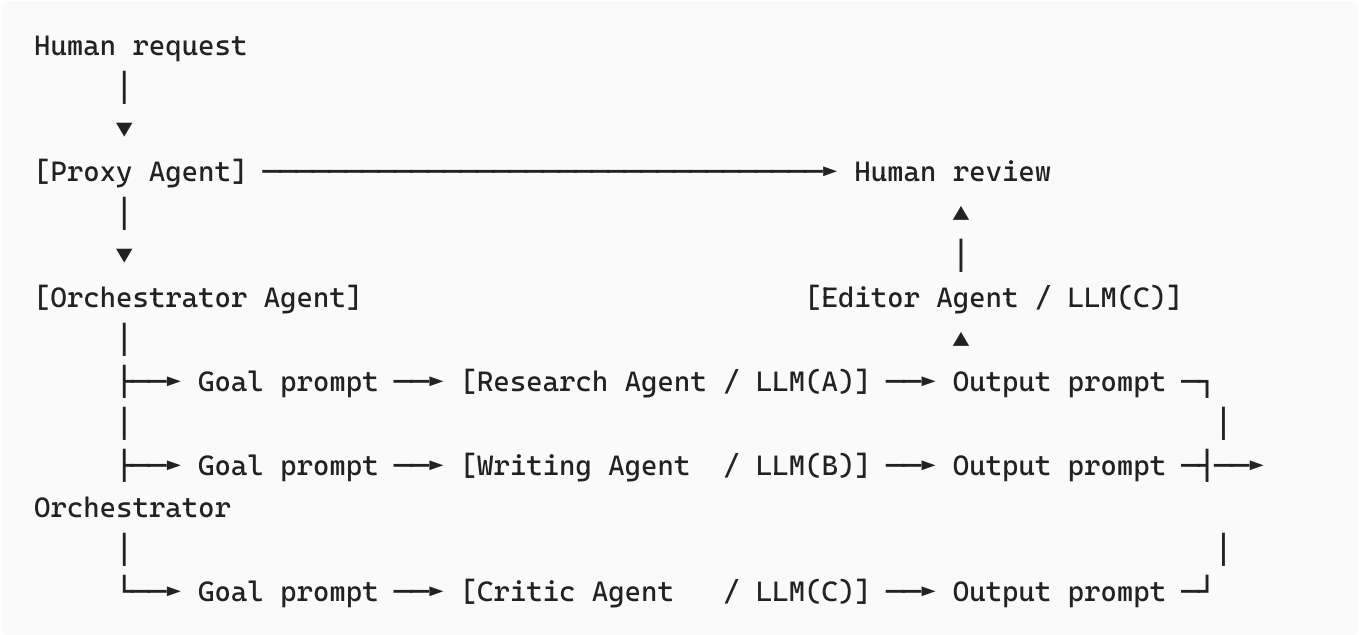


Arquitectura MultiAgente: AI Agent Blog Writer (Gartner)

Fuente: Gartner 817826_C. Este diagrama es uno de los ejemplos de referencia más citados para explicar qué es un sistema MultiAgent real y cómo se diferencia de un simple agente.

Visión general del sistema

El sistema recibe una solicitud humana ("escribe un blog sobre X") y produce un artículo revisado, pasando por una cadena de agentes especializados que investigan, escriben, critican y editan — cada uno con su propio LLM, sus propios prompts de objetivo y su propia responsabilidad.



Leyenda: los tres tipos de nodos

Símbolo	Tipo	Qué es
Rectángulo azul sólido	Prompt	Instrucción o mensaje que entra o sale de un agente
Caja punteada con chip	Agent	Entidad autónoma con LLM propio que razona y actúa

Símbolo	Tipo	Qué es
Hexágono oscuro (LLM X)	AI Technique	El modelo de lenguaje específico que usa ese agente

Los 8 pasos anotados en la imagen

Paso 1 — User Prompt: el punto de entrada

El humano formula su solicitud en lenguaje natural: "escribe un artículo de blog sobre tendencias de IA en 2025".

Este prompt llega al **Proxy Agent** — el primer agente del sistema.

Por qué no va directamente al Orchestrator: el Proxy actúa como capa de traducción entre el lenguaje informal del humano y el lenguaje estructurado que necesita el orquestador. Puede clarificar ambigüedades, validar el pedido y estructurarlo antes de pasarlo al interior del sistema.

Paso 2 — Proxy Agent con LLM(Y): comunicación con el humano

El **Proxy Agent** es el único agente del sistema que tiene contacto directo con el humano en ambas direcciones:

- Recibe la solicitud inicial del humano
- Devuelve el resultado final al humano para revisión
- Puede recibir feedback del humano y reinyectarlo al sistema

LLM(Y): usa un modelo optimizado para comunicación natural con usuarios — conversacional, claro, capaz de manejar ambigüedad.

Responsabilidades del Proxy:

1. Entender la intención real detrás del pedido
2. Estructurar el objetivo para el Orchestrator
3. Gestionar el feedback del humano si el resultado no satisface
4. Presentar el output final de forma amigable

Analogía: el Proxy es la recepcionista de una consultoría — no hace el trabajo técnico, pero es quien habla con el cliente y asegura que el trabajo interno tenga el contexto correcto.

Paso 3 — Orchestrator Agent con LLM(Y): coordinación de capacidades

El **Orchestrator Agent** es el cerebro central del sistema. Recibe el objetivo estructurado del Proxy y decide:

- Qué subagentes necesita convocar
- Qué objetivo específico (Goal Prompt) darle a cada uno
- En qué orden convocarlos
- Cómo integrar sus outputs en un resultado coherente

LLM(Y): el mismo modelo que el Proxy — esto no es casualidad. El Orchestrator y el Proxy comparten capacidades de razonamiento de alto nivel porque ambos necesitan entender el objetivo global, no ejecutar una tarea específica.

Lo que hace el Orchestrator autónomamente:

- Descompone "escribe un blog" en subproblemas: investigar → redactar → criticar → editar
- Genera un Goal Prompt específico para cada subagente
- Recibe los Output Prompts de cada subagente
- Decide si los resultados son suficientes o si necesita invocar agentes adicionales
- Pasa el resultado consolidado al Editor Agent

Por qué el Orchestrator no hace el trabajo él mismo: separar coordinación de ejecución es un principio de diseño clave. El Orchestrator que intenta también investigar, escribir y criticar al mismo tiempo pierde calidad en todo. La especialización permite optimizar cada rol con el LLM y los prompts más adecuados.

Paso 4 — Goal Prompt: el subobjetivo como prompt

Cuando el Orchestrator convoca a un subagente, lo hace enviándole un **Goal Prompt**: una instrucción específica que define exactamente qué debe producir ese agente.

El Goal Prompt **no es el User Prompt original** — es una transformación. El Orchestrator traduce el objetivo global en subobjetivos concretos:

Agente receptor	Ejemplo de Goal Prompt generado por el Orchestrator
Research Agent	"Encuentra los 5 avances más relevantes en IA generativa publicados en los últimos 6 meses. Incluye fuentes, fechas y descripción de 2-3 oraciones por avance."
Writing Agent	"Usando la investigación adjunta, escribe un artículo de blog de 800 palabras con tono profesional pero accesible. Estructura: introducción, 3 secciones temáticas, conclusión."
Critic Agent	"Evalúa el siguiente borrador de blog. Identifica: debilidades argumentales, afirmaciones sin respaldo, problemas de fluidez y oportunidades de mejora. Sé específico y riguroso."

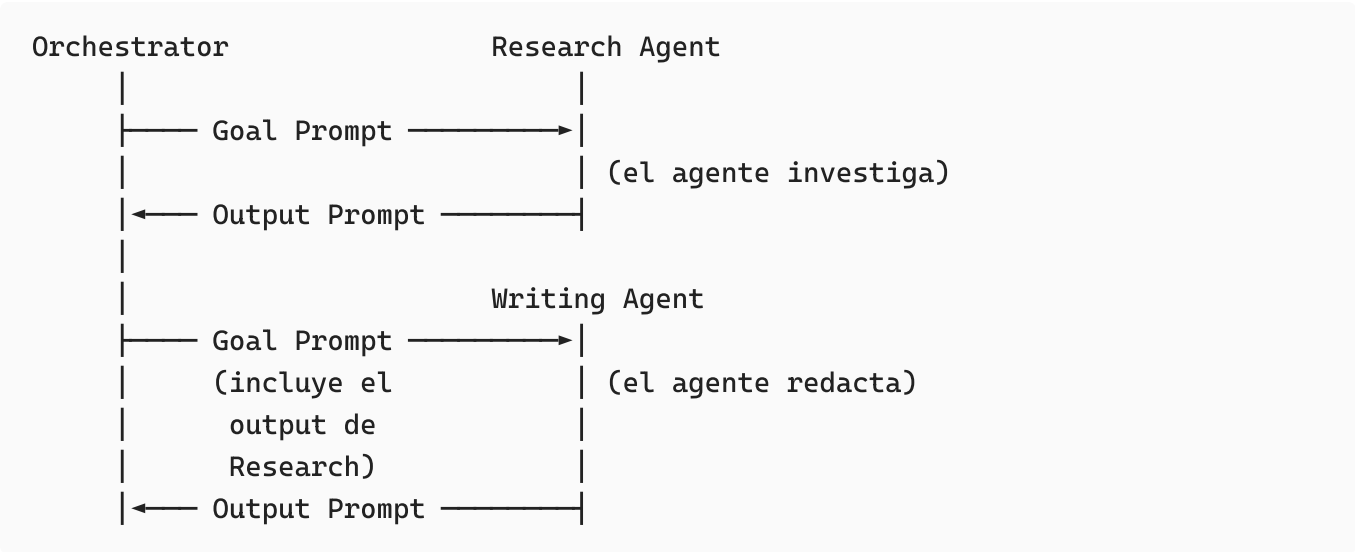
Esto es orquestación del plan de resolución: el Orchestrator genera dinámicamente cada Goal Prompt según el contexto — no son prompts hardcodeados de antemano.

Paso 5 — Output Prompt: el subagente devuelve su resultado

Cada subagente completa su tarea y devuelve un **Output Prompt** al Orchestrator: su resultado estructurado, listo para ser usado por el siguiente paso.

El Output Prompt no es texto libre — el subagente lo formatea según lo que el Orchestrator necesita recibir. Esta interfaz estandarizada es lo que permite al Orchestrator integrar outputs de múltiples agentes heterogéneos.

Flujo de datos:



El Orchestrator puede pasar el output de un agente como input del siguiente — esto es **encadenamiento dinámico de agentes**.

Paso 6 — Agentes en un MAS pueden usar diferentes LLMs

El **Writing Agent** usa **LLM(B)** — un modelo distinto al LLM(Y) del Orchestrator y al LLM(A) del Research Agent.

Esta es una característica definitoria de los sistemas MultiAgent maduros: **cada agente usa el LLM más adecuado para su tarea específica**.

Agente	LLM	Por qué ese modelo
Proxy Agent	LLM(Y)	Conversacional, bueno entendiendo lenguaje informal
Orchestrator Agent	LLM(Y)	Razonamiento de alto nivel, descomposición de objetivos
Research Agent	LLM(A)	Optimizado para síntesis de información y búsqueda factual
Writing Agent	LLM(B)	Optimizado para generación de texto fluido y creativo
Critic Agent	LLM(C)	Capacidad de análisis crítico, detección de fallos
Editor Agent	LLM(C)	Refinamiento fino del lenguaje, consistencia de estilo

Implicación práctica: en un sistema real, LLM(A) podría ser un modelo pequeño y rápido para búsqueda, LLM(B) un modelo grande y creativo para escritura, y LLM(C) un modelo con fuerte capacidad analítica para crítica y edición. Usar el modelo correcto en cada rol optimiza calidad y costo simultáneamente.

Paso 7 — Critic Agent con LLM(C): self-reflection y self-critique

El **Critic Agent** es uno de los patrones más importantes en sistemas MultiAgent: un agente cuya única función es evaluar críticamente el trabajo de otros agentes.

¿Por qué no puede el Writing Agent criticar su propio texto? Por la misma razón que un escritor humano necesita un editor: quien produce el contenido tiene sesgos cognitivos hacia su propio output. Un agente separado, sin el contexto de "yo lo escribí", evalúa con mayor objetividad.

Lo que hace el Critic Agent:

- Lee el borrador producido por el Writing Agent

- Identifica afirmaciones no respaldadas por la investigación
- Detecta inconsistencias argumentales
- Señala problemas de estructura o fluidez
- Propone mejoras específicas con referencia al texto original

Su output NO es el texto corregido — es un reporte de crítica. La corrección la hace el Editor Agent o el Writing Agent en una segunda iteración.

Self-reflection en sistemas MultiAgent: este patrón (agente que critica a otros agentes) es la base de la auto-mejora en sistemas de IA. Sin él, el sistema produce y entrega sin verificar calidad.

Paso 8 — Editor Agent + Human review: colaboración humano-IA

El **Editor Agent con LLM(C)** es el último nodo antes de que el resultado llegue al humano. Recibe:

- El borrador mejorado (post-crítica)
- Las anotaciones del Critic Agent
- Posiblemente el feedback previo del humano si hubo iteraciones

Su función: hacer el refinamiento final — consistencia de tono, estilo, longitud, formato — para que el texto esté listo para publicación.

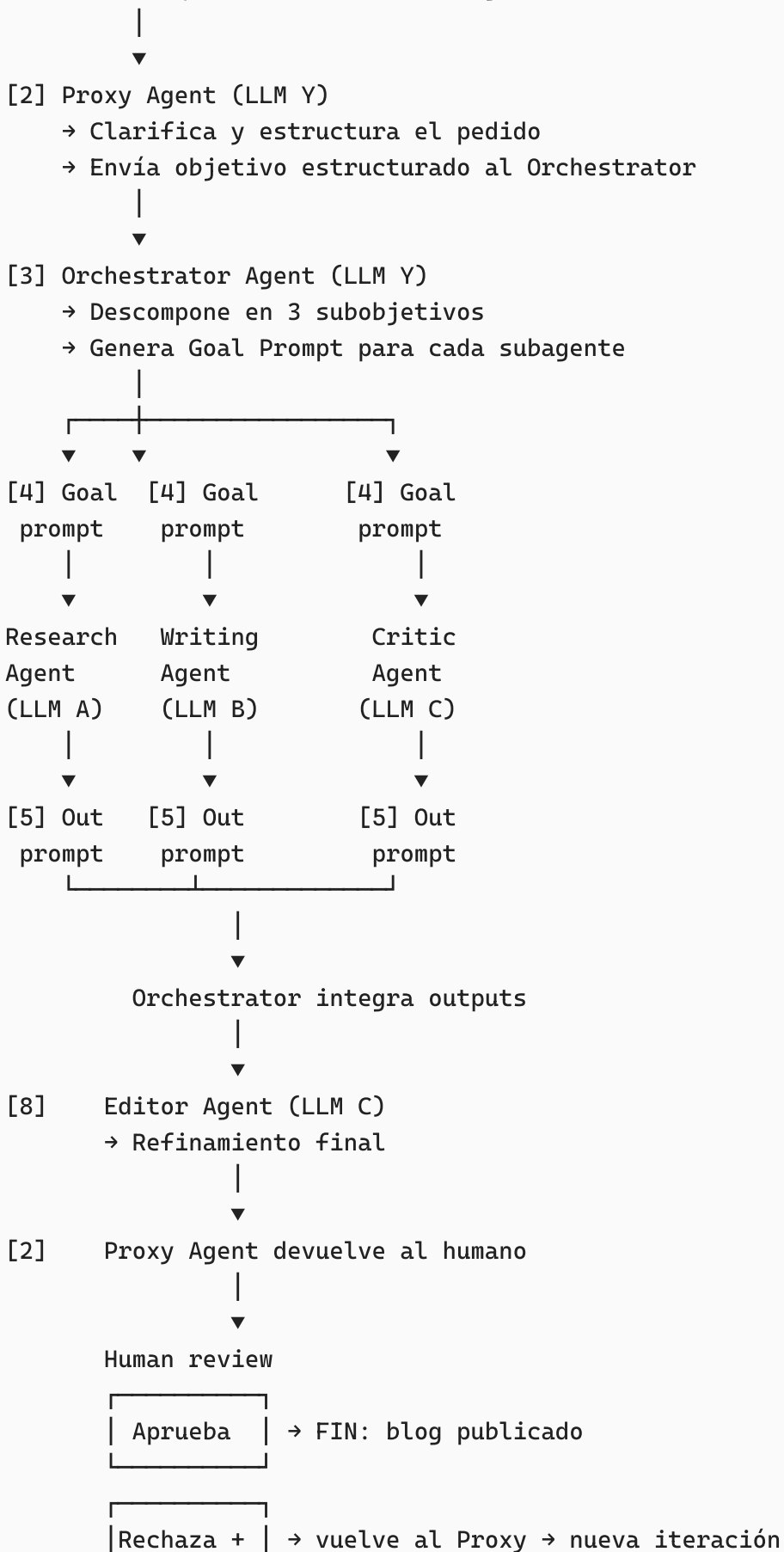
Human review recibe el output del Editor Agent. Esta es la etapa de supervisión humana del sistema. El humano puede:

1. Aprobar el resultado → el blog está listo
2. Rechazarlo con feedback → el feedback vuelve al Proxy Agent y se inicia una nueva iteración

"Human and AI agent coworker relationship": el Gartner diagram nombra explícitamente esta relación como "compañero de trabajo" — no "herramienta". El humano no controla cada paso del proceso; revisa el resultado final y guía las iteraciones. La IA hace el trabajo pesado; el humano aporta juicio final.

Flujo completo del sistema

```
[1] Human request: "Escribe un blog sobre tendencias de IA 2025"
```



Por qué ESTO es MultiAgent y no solo un Agente

Esta es la pregunta central. Veamos qué lo convierte en MultiAgent:

Criterio 1 — Múltiples entidades autónomas con LLM propio

No hay un solo LLM que hace todo. Hay **6 agentes distintos**, cada uno con su propio LLM, sus propios prompts y su propia responsabilidad:

```
Proxy (LLM Y) + Orchestrator (LLM Y) + Research (LLM A)
+ Writing (LLM B) + Critic (LLM C) + Editor (LLM C)
```

Un agente único con un solo LLM que intenta hacer todo esto sería como contratar a una persona para que sea simultáneamente investigador, redactor, editor y gerente de proyecto. La calidad en cada rol sería inferior a tener especialistas.

Criterio 2 — Especialización de roles

Cada agente tiene un dominio de conocimiento, un estilo de prompt y un LLM optimizados para una tarea concreta:

Si fuera un agente único...	Al ser MultiAgent...
El mismo LLM investiga y luego escribe — los sesgos de la investigación contaminan la redacción	Research Agent entrega datos limpios; Writing Agent los interpreta sin haberlos buscado
El mismo LLM critica lo que acaba de escribir — dificultad para detectar sus propios errores	Critic Agent evalúa con distancia cognitiva real
El mismo LLM coordina y ejecuta — sobrecarga de contexto	Orchestrator solo coordina; los subagentes solo ejecutan

Criterio 3 — Orquestación explícita del plan

El **Orchestrator Agent** existe exclusivamente para coordinar. Esta es la capa de orquestación del plan de resolución (rectángulo naranja del slide anterior): el Orchestrator crea dinámicamente el plan, delega a subagentes y recibe resultados.

En un agente simple, este rol de coordinación y el rol de ejecución están mezclados en el mismo LLM — lo que degrada la calidad de ambos.

Criterio 4 — Bucle de self-critique entre agentes

El Critic Agent (paso 7) introduce un **bucl**e de retroalimentación entre agentes: un agente evalúa el output de otro. Esto no es posible en un agente único sin "dividir" artificialmente al LLM, lo cual es ineficiente.

En un sistema MultiAgent, este bucle es natural: el Critic Agent simplemente recibe el output del Writing Agent y devuelve una evaluación al Orchestrator.

Criterio 5 — Human-in-the-loop en punto de control específico

El humano no interviene en cada paso — solo al final (paso 8). El sistema MultiAgent permite **encapsular la autonomía** y exponer al humano solo el resultado final, con un mecanismo de feedback que re-ingresa al sistema sin interrumpir el flujo interno.

Los criterios para pasar de Workflow → Agent → MultiAgent

¿Cuándo el Workflow ya no es suficiente? → Necesitas un Agent

Un sistema de tipo Workflow deja de ser suficiente cuando se presenta cualquiera de estas condiciones:

Señal de alarma en el Workflow	Lo que necesitas
Los pasos del flujo cambian según el resultado de pasos anteriores	Replanning dinámico → Agent
La tarea tiene caminos posibles que no puedes enumerar de antemano	Plan autónomo → Agent
El sistema necesita decidir cuándo terminar, no solo ejecutar hasta el último paso	Loop con condición de salida → Agent
La selección de herramientas depende del contexto actual, no de reglas fijas	Tool selection autónoma → Agent
El flujo falla en silencio cuando ocurre algo inesperado	Capacidad de recuperación → Agent

Regla simple: si puedes dibujar el diagrama de flujo completo antes de ejecutar el sistema, puede ser un Workflow. Si el diagrama solo lo puedes dibujar después de que el sistema corrió, necesitas un Agent.

¿Cuándo el Agent ya no es suficiente? → Necesitas MultiAgent

Un agente único llega a su límite cuando aparece cualquiera de estas condiciones:

Condición 1 — Límite de contexto (context window)

Un LLM tiene un límite de tokens que puede procesar en una sola llamada. Si la tarea requiere:

- Investigar 50 artículos
- Escribir 3000 palabras
- Criticar y editar el resultado

...todo eso junto excede el contexto de cualquier modelo. La solución es dividir en agentes especializados, cada uno con su propio contexto limpio.

Agent único:

[investigación: 10k tokens] + [borrador: 5k tokens] + [crítica: 3k tokens] = 18k tokens

→ degradación de calidad cuando se acerca al límite

MultiAgent:

Research Agent: 10k tokens propios ✓

Writing Agent: 5k tokens propios ✓

Critic Agent: 3k tokens propios ✓

Condición 2 — Dominios de conocimiento incompatibles

Si la tarea requiere expertise en áreas muy distintas simultáneamente, un único agente con un único system prompt no puede optimizar para todas:

- Un agente que debe ser experto legal Y experto financiero Y experto técnico tendrá system prompts en conflicto o demasiado largos para ser efectivos
- La solución: un agente por dominio, cada uno con su propio system prompt especializado

Condición 3 — Paralelismo posible

Si partes de la tarea son independientes entre sí, ejecutarlas secuencialmente en un solo agente es ineficiente:

Agent único (secuencial):

Research [3 min] → Write [2 min] → Critique [1 min] → Edit [1 min] = 7 min total

```
MultiAgent (paralelo donde es posible):
Research [3 min] ┌
                  └─> Orchestrator integra → Write → Critique → Edit = ~5 min
total
(preparación)  [1m]┐
```

Condición 4 — Necesitas self-critique estructurada

Un agente único puede "autocriticarse" en el mismo prompt, pero el LLM tiene sesgo hacia validar su propio output. Un Critic Agent separado, sin haber "escrito" el texto, evalúa con mayor objetividad porque no tiene el contexto emocional de la creación.

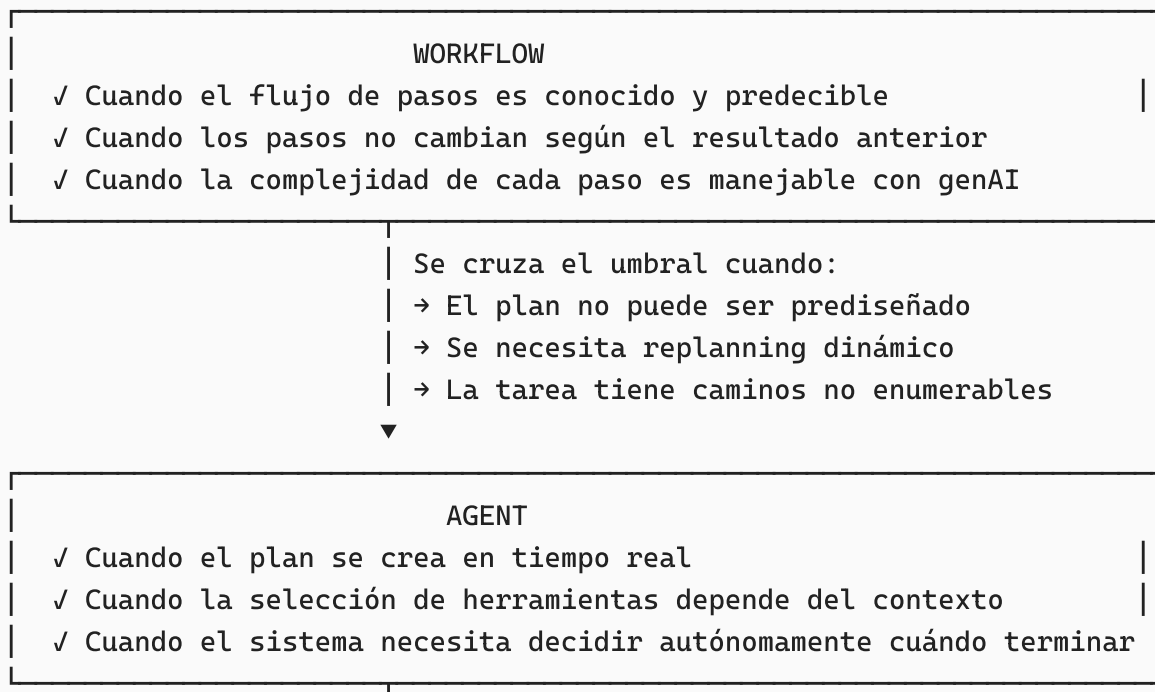
Condición 5 — Diferentes LLMs son óptimos para diferentes subtareas

Si una subtarea requiere creatividad (mejor con modelos grandes), otra requiere velocidad (mejor con modelos pequeños) y otra requiere análisis estructurado (mejor con modelos con reasoning), un único agente no puede tener tres LLMs. Un sistema MultiAgent sí.

Condición 6 — Escalabilidad y mantenimiento

Si necesitas mejorar solo el componente de escritura sin tocar la investigación ni la crítica, en un agente único no puedes hacerlo sin riesgo de degradar todo. En MultiAgent, actualizas solo el Writing Agent.

Resumen de los criterios de transición



| Se cruza el umbral cuando:
| → El contexto del LLM resulta insuficiente
| → La tarea requiere expertise en dominios distintos
| → Las subtarefas pueden ejecutarse en paralelo
| → Se necesita self-critique estructurada
| → Distintos LLMs son óptimos para distintas partes



MULTIAGENT

- ✓ Orquestador crea el plan y delega a agentes especializados
- ✓ Cada agente tiene su propio LLM, tools, knowledge y memory
- ✓ Self-critique entre agentes (no dentro del mismo agente)
- ✓ Paralelismo real entre subagentes
- ✓ Human-in-the-loop en punto de control específico

Patrones de diseño visibles en esta arquitectura

Patrón 1 — Proxy Pattern

El Proxy Agent separa la interfaz con el humano del motor interno del sistema. El sistema puede cambiar completamente su arquitectura interna sin que el humano note diferencia en cómo interactúa.

Patrón 2 — Orchestrator-Subagent Pattern

Un agente central coordina; los subagentes ejecutan. El Orchestrator nunca hace trabajo de dominio; los subagentes nunca hacen coordinación. Separación de responsabilidades limpia.

Patrón 3 — Critic / Reflection Pattern

Un agente dedicado a evaluar el output de otros introduce calidad sistémica sin depender de la autocrítica (que tiene sesgos) del agente productor.

Patrón 4 — Human-in-the-Loop con feedback reinyectado

El humano no interrumpe el flujo interno — solo interviene al final. Su feedback no se pierde: vuelve al Proxy y se convierte en input para una nueva iteración completa.

Patrón 5 — Heterogeneidad de LLMs

Usar diferentes modelos para diferentes roles es una decisión de optimización: calidad donde importa (Writing, Critic), velocidad donde alcanza (Research), coordinación general

(Orchestrator).

Comparación: ¿cómo luciría esto como Workflow y como Agent único?

Como Workflow

Paso 1 (fijo): buscar información en web sobre IA 2025
Paso 2 (fijo): generar borrador con la información del paso 1
Paso 3 (fijo): aplicar plantilla de formato
Paso 4 (fijo): enviar al humano

Limitaciones:

- No puede ajustar la profundidad de investigación según el tema
- No puede decidir si el borrador necesita más investigación
- No tiene critic – el borrador va directo al humano sin revisión
- Si el paso 1 falla, el pipeline entero falla

Como Agent único

Agente único (un LLM, un contexto):

- Investiga Y redacta Y critica Y edita en el mismo contexto
- El contexto se llena rápidamente con toda la información
- La calidad de escritura se ve afectada por el ruido de la investigación
- La autocritica tiene sesgo hacia validar lo que el mismo LLM produjo
- Solo puede usar un LLM – no puede optimizar por subtask

Como MultiAgent (la imagen)

Cada agente tiene contexto limpio y responsabilidad única

- Research Agent: solo investiga, con LLM óptimo para búsqueda factual
- Writing Agent: recibe información limpia, escribe sin ruido de contexto
- Critic Agent: evalúa sin haber "creado" el texto → más objetivo
- Editor Agent: refinamiento fino antes de entregar al humano
- Proxy + Orchestrator: coordinación separada de ejecución

Relación con los conceptos anteriores

- Este sistema es el **Nivel 4 — MultiAgents** del espectro. Ver [espectro_llm_workflow_agent_multiagent.md](#)
- Cada subagente (Research, Writing, Critic, Editor) corresponde al **Tipo 6** del cuadro de Conversational Automation: tiene System Prompt + Context + Input Prompt propios. Ver [taxonomia_conversational_automation.md](#)
- Cada agente cumple los 4 verbos de Gartner (percibir, decidir, actuar, lograr objetivos) dentro de su dominio. Ver [../Modulo1_Introduccion_Motivacion/Sesion01_Introduccion_Agentes/que_es_un_agente_ia.md](#)
- Las ideas de proyecto final están en el nivel **Agent**. Si alguna crece en complejidad, este diagrama muestra cómo escalarla a **MultiAgent**. Ver [ideas_proyecto_final_agent_profile_cards.md](#)